

Seleção V-LAB - Desafio Técnico Full Stack

Solicitações de Atendimento

1. Objetivo

Este desafio foi elaborado para avaliar pessoas candidatas às vagas de desenvolvimento full stack do V-Lab. O objetivo é observar a capacidade de construir uma solução funcional de ponta a ponta com React, PHP/Laravel e PostgreSQL, integrar frontend, backend e banco de dados, organizar o código por responsabilidades e domínio, justificar decisões técnicas e entregar um ambiente reproduzível.

Nível esperado: pessoas candidatas a bolsas de desenvolvimento full stack e a uma vaga de analista júnior. Todas receberão o mesmo desafio, mas a avaliação considerará o nível de autonomia e responsabilidade esperado para cada perfil.

2. O Desafio

No contexto de uma solução para a área da saúde pública, deseja-se uma aplicação para registrar e acompanhar solicitações de atendimento encaminhadas a unidades públicas. Sua missão é desenvolver uma aplicação full stack que permita criar, consultar, filtrar e atualizar essas solicitações de forma simples e confiável, mantendo frontend e backend desacoplados por uma API REST.

A solução deverá usar apenas dados fictícios. Não utilize informações médicas reais, documentos pessoais reais ou qualquer dado sensível de terceiros.

2.1 - Stack obrigatória

A entrega deve utilizar obrigatoriamente React com TypeScript no frontend, PHP com Laravel no backend e PostgreSQL como banco de dados. Docker e Docker Compose devem permitir a execução integrada da solução. Bibliotecas complementares são livres, desde que sejam compatíveis com a stack, justificadas no README e não substituam as tecnologias obrigatórias.

Camada	Tecnologia	Observações
Frontend	React com TypeScript	Implementar a interface como aplicação React tipada.
Backend	PHP com Laravel	Construir uma API REST com Laravel.
Banco de dados	PostgreSQL	Persistência relacional com migrations do Laravel.
Infraestrutura	Docker e Docker Compose	Subida integrada do frontend, da API Laravel e do PostgreSQL.

2.2 - Requisitos obrigatórios para validade da entrega (eliminatórios)

A entrega somente seguirá para pontuação quando atender aos requisitos mínimos abaixo:

- Utilizar React com TypeScript no frontend, PHP com Laravel no backend e PostgreSQL como banco de dados.
- Apresentar integração real entre o frontend React, a API Laravel e o PostgreSQL; dados estáticos não podem substituir a integração principal.
- Implementar ao menos um fluxo ponta a ponta que inclua criação, consulta ou listagem e atualização de status.
- Persistir os dados no PostgreSQL por meio do backend Laravel.
- Respeitar as regras obrigatórias de protocolo, prioridade urgente e transição de status descritas neste documento.
- Fornecer instruções suficientes para executar e avaliar a solução, incluindo configurações e migrations necessárias.
- Entregar Docker Compose para a execução integrada do frontend, da API e do banco de dados.
- Utilizar somente dados fictícios e não versionar senhas, tokens, credenciais ou outros segredos reais.

Uma falha pontual e facilmente corrigível na inicialização do ambiente não invalida automaticamente a entrega quando houver instruções claras e uma forma razoável de execução. A ausência de execução reproduzível, da stack obrigatória ou de integração real invalida a entrega.

2.3 - Requisitos funcionais pontuáveis

As funcionalidades a seguir compõem a avaliação de completude e qualidade. Além do fluxo mínimo exigido para validade, funcionalidades não concluídas reduzem a pontuação, mas não invalidam a entrega quando forem informadas com transparência no README.

A. Modelo de dados

Cada solicitação deverá possuir, no mínimo:

- id: identificador único.
- protocolo: código único gerado pela aplicação.
- nome_solicitante: nome fictício da pessoa solicitante.
- categoria: CONSULTA, EXAME, VACINACAO ou OUTRO.
- prioridade: BAIXA, MEDIA, ALTA ou URGENTE.
- status: RECEBIDA, EM_ANALISE, AGENDADA, CONCLUIDA ou CANCELADA.
- descricao: resumo da solicitação.
- justificativa_prioridade: obrigatória quando a prioridade for URGENTE.
- data_criacao e data_atualizacao.

B. Regras de negócio

- Toda solicitação deve ser criada com status inicial RECEBIDA.
- Toda solicitação deve possuir um protocolo único, gerado automaticamente pela aplicação.
- Solicitações com prioridade URGENTE devem possuir justificativa de prioridade preenchida.
- A data de criação deve ser gerada automaticamente no cadastro.
- A data de atualização deve ser modificada sempre que a solicitação for atualizada.
- Requisições inválidas devem retornar mensagens de erro claras e códigos HTTP adequados.
- A alteração de status deve respeitar o fluxo abaixo:

Status atual	Próximos status permitidos	Observação
RECEBIDA	EM_ANALISE, CANCELADA	Solicitação recém-criada pode entrar em análise ou ser cancelada.
EM_ANALISE	AGENDADA, CANCELADA	Após análise, pode ser agendada ou cancelada.
AGENDADA	CONCLUIDA, CANCELADA	Após agendamento, pode ser concluída ou cancelada.
CONCLUIDA	Nenhum	Status final. Não permite nova alteração.
CANCELADA	Nenhum	Status final. Não permite nova alteração.

C. Backend - API REST

Método e rota sugerida	Comportamento esperado
POST /api/v1/solicitacoes	Criar uma nova solicitação.
GET /api/v1/solicitacoes	Listar com paginação e filtros por status, categoria e prioridade.
GET /api/v1/solicitacoes/{id}	Consultar os detalhes de uma solicitação.
PATCH /api/v1/solicitacoes/{id}/status	Atualizar o status, respeitando as regras de transição.

As rotas abaixo são sugestões de padronização para a API REST em Laravel. Alterações são permitidas desde que sejam consistentes, documentadas e preservem as funcionalidades solicitadas.

D. Frontend

- Tela inicial com resumo das solicitações por status ou prioridade.
- Listagem paginada das solicitações.
- Filtros por status, categoria e prioridade.
- Formulário para criação de uma solicitação, com validação dos campos.
- Visualização dos detalhes de uma solicitação.
- Ação para atualização de status.
- Estados visuais de carregamento, sucesso, vazio e erro.
- A interface deve ser implementada em React com TypeScript, apresentar layout minimamente responsivo e navegação compreensível.

E. Integração e infraestrutura

- O frontend React deve consumir exclusivamente a API Laravel para as operações do desafio, sem depender de dados estáticos como fonte principal.
- O frontend, a API Laravel e o PostgreSQL devem poder ser executados de forma integrada com Docker Compose.
- As configurações variáveis devem ser fornecidas por variáveis de ambiente.
- O repositório deve conter um arquivo de exemplo, como .env.example, sem segredos reais.
- O PostgreSQL deve ser inicializado de maneira reproduzível por meio de migrations do Laravel; seeders ou factories podem ser usados para dados fictícios.

2.4. Requisitos técnicos pontuáveis e arquitetura

A. Backend

- Utilizar os recursos do Laravel para validação rigorosa dos dados de entrada, preferencialmente Form Requests ou abordagem equivalente e consistente.

- Demonstrar separação clara de responsabilidades. Controllers não devem concentrar regras de negócio relevantes, e as transições de status devem possuir implementação central, consistente e testável.
- Tratar exceções e respostas HTTP de forma consistente, com mensagens úteis para o consumidor da API e sem exposição desnecessária de detalhes internos.
- Utilizar Eloquent, dependency injection, transações, services, actions ou outras estruturas quando trouxerem clareza e consistência. Políticas são esperadas somente quando houver autorização. Abstrações e camadas sem benefício demonstrável poderão ser avaliadas negativamente.
- Manter no repositório uma especificação OpenAPI que descreva, no mínimo, endpoints implementados, parâmetros e filtros, corpos de requisição, respostas de sucesso, erros de validação e principais códigos HTTP. Swagger UI é recomendada, mas não obrigatória, e não é exigida geração automática da especificação.

B. Frontend

- Implementar o frontend em React com componentes coesos, responsabilidades bem definidas e organização por funcionalidades ou domínio.
- Utilizar TypeScript para tipar os dados consumidos e enviados à API, evitando o uso indiscriminado de any e mantendo contratos compreensíveis entre frontend e backend.
- Formulários com validação e mensagens claras para a pessoa usuária.
- Organizar de forma consistente o cliente HTTP e o tratamento dos estados assíncronos, incluindo carregamento, sucesso, vazio e erro.
- Cuidados básicos de semântica HTML, acessibilidade e responsividade.

C. Banco de dados

- Modelar os dados de forma relacional e coerente com o domínio no PostgreSQL.
- Definir restrições de integridade para campos únicos e obrigatórios, além de índices adequados aos filtros propostos. Decisões sobre enums, constraints e índices devem ser coerentes e explicáveis.
- Versionar o esquema exclusivamente por migrations do Laravel. Alterações manuais no banco não devem ser necessárias para executar a solução.

D. Testes e qualidade

- Implementar testes automatizados com PHPUnit ou Pest para pelo menos uma regra de negócio relevante do backend Laravel, priorizando cenários que demonstrem comportamento e não apenas cobertura de linhas.
- Implementar pelo menos um teste relevante no frontend React, com Vitest/Jest e React Testing Library, ou um teste de integração ponta a ponta com Cypress/Playwright.
- Garantir que os testes sejam determinísticos, independentes de dados reais e executáveis conforme as instruções do README.

- Ferramentas de lint e formatação adequadas a PHP/Laravel e React/TypeScript são recomendadas.

E. Segurança e privacidade

- Não versionar senhas, tokens ou credenciais.
- Não utilizar dados pessoais ou clínicos reais.
- Validar entradas e evitar exposição desnecessária de detalhes internos nos erros.
- Caso implemente autenticação, utilizar mecanismo compatível com Laravel e documentar perfis, autorização e decisões adotadas.

2.5. Diferenciais técnicos - Bônus - Até 20 pontos adicionais

Os diferenciais podem elevar a pontuação final acima de 100 pontos, mas não substituem os requisitos eliminatórios nem os mínimos de frontend e backend. A presença de ferramentas, infraestrutura ou abstrações sem utilidade demonstrável não gera pontuação automaticamente.

- Autenticação e autorização simples em Laravel, com diferenciação de perfis e uso consistente de policies ou mecanismo equivalente.
- Health check da API Laravel e verificação da conexão com o PostgreSQL.
- Migrations do Laravel executadas automaticamente na inicialização do ambiente, com seeders ou factories para dados fictícios.
- Logs estruturados, identificação/correlação de requisições e tratamento adequado de falhas de integração.
- Pipeline de integração contínua para lint, testes e build do Laravel e do React.
- Documentação arquitetural com diagrama simples e registro das decisões, incluindo limites entre frontend, API e banco e uma breve explicação de como o módulo poderia evoluir ou integrar-se a uma arquitetura de serviços por domínio.
- Melhorias de acessibilidade e experiência da pessoa usuária.
- Dados iniciais fictícios para facilitar a avaliação.
- Uso pertinente de eventos, filas ou processamento assíncrono do Laravel, com justificativa e tratamento de falhas.

3. Política de uso de inteligência artificial

- O uso de ferramentas de IA é permitido e encorajado, sendo obrigatório indicar no README como utilizou essas ferramentas e em quais partes do projeto.
- Todo código entregue deve ser compreendido e explicável pela pessoa candidata.
- Na entrevista técnica, poderão ser solicitadas explicações ou pequenas alterações no projeto.

4. Entrega

- Prazo: 7 dias corridos após o recebimento do desafio.
- O histórico de commits será considerado como evidência de organização do trabalho, mas não haverá exigência de quantidade mínima.

4.1. O README deve conter, no mínimo:

- Tecnologias e versões utilizadas, incluindo React, TypeScript, PHP, Laravel e PostgreSQL;
- Instruções completas para executar frontend, API Laravel, PostgreSQL, migrations e seeders com Docker Compose;
- Principais decisões arquiteturais, organização por domínio/responsabilidade, contratos entre frontend e API e justificativa das abstrações utilizadas;
- Funcionalidades implementadas, não implementadas, limitações conhecidas e eventuais ajustes necessários para execução;
- Como executar os testes e onde localizar a especificação OpenAPI;
- Descrição do uso de ferramentas de inteligência artificial.

5. Critérios de Avaliação

Critério	Pontos	O que será observado
Funcionamento e integração ponta a ponta	25	Fluxo principal executável (10); integração real entre camadas (6); funcionalidades entregues (5); estabilidade do fluxo (4).
Backend	15	Regras de negócio (4); API REST (3); uso coerente do Laravel (3); PostgreSQL e persistência (3); validação e erros (2).
Frontend	15	React e TypeScript (4); componentização (3); formulários (3); estados e UX (3); responsividade e acessibilidade (2).
Organização e arquitetura	15	Separação de responsabilidades (5); legibilidade (3); organização por domínio/funcionalidade (3); decisões justificadas (2); proporcionalidade das abstrações (2).
Testes e confiabilidade	15	Testes de backend (6); frontend ou E2E (4); relevância dos cenários (3); execução, isolamento e determinismo (2).

Validação, segurança e privacidade	10	Validação de entradas (4); respostas seguras (2); proteção de credenciais (2); uso exclusivo de dados fictícios (2).
Documentação e execução	5	README e execução (2); Docker Compose (1); OpenAPI (1); migrations e reprodutibilidade (1).
Bônus (pontuação extra)	20	Até 20 pontos adicionais por diferenciais pertinentes e bem implementados. Não substitui requisitos obrigatórios ou mínimos por camada.
Pontuação-base máxima	100	Soma dos critérios obrigatórios e pontuáveis, sem bônus.
Pontuação total possível	120	Até 100 pontos de base e até 20 pontos adicionais de bônus.

Critérios mínimos

A pessoa candidata deverá obter pelo menos 60 dos 100 pontos da avaliação-base, além de no mínimo 6 dos 15 pontos de backend e 6 dos 15 pontos de frontend. O bônus pode elevar a nota acima de 100, mas não compensa a entrega inválida, ausência de uma camada ou o não atendimento aos mínimos por camada.

5.1. Interpretação por perfil e observações

O desafio e a matriz de pontos são os mesmos para todas as pessoas candidatas. A interpretação considera o nível de autonomia e responsabilidade esperado para o perfil da vaga.

A. Pessoas candidatas a bolsas

- Serão observados principalmente fundamentos, integração entre as camadas, organização, compreensão do próprio código, qualidade das validações, teste de uma regra relevante e capacidade de reconhecer limitações.
- Não se espera domínio avançado de arquitetura distribuída, observabilidade ou segurança, embora diferenciais pertinentes possam gerar bônus.

B. Pessoa candidata a analista júnior

- Além dos fundamentos, serão esperados maior autonomia, tratamento consistente de erros, separação de responsabilidades e modelagem mais cuidadosa.
- Serão valorizados contratos claros entre frontend e backend, testes com cenários relevantes, documentação reproduzível e decisões técnicas proporcionais e justificadas.

5.2. Considerações finais

Priorize qualidade e transparência

Os requisitos eliminatórios devem ser atendidos, mas não é obrigatório concluir todos os requisitos pontuáveis.

Por se tratar de uma vaga full stack, a entrega deve apresentar ao menos um fluxo funcional integrado entre frontend, backend e banco.

Priorize a qualidade do que for entregue: é melhor apresentar um fluxo ponta a ponta bem implementado, integrado e estável, com limitações claramente documentadas, do que muitas telas e endpoints com bugs ou código desorganizado.

Boa sorte!